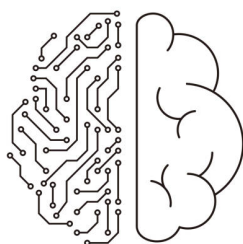




UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Word-Embeddings and Feed-Forward Neural Networks in NLP



Student name: Antonis Zikas

email: sdi2100038@di.uoa.gr

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1	Data Processing and Analysis	2
1.1	Data Pre-processing	2
1.1.1	General Cleaning	2
1.1.2	Stop-words Removal	2
1.2	Exploratory Data Analysis	3
1.2.1	Sentiment Correlation	3
1.2.2	Dataset Splitting	3
1.2.3	Sentiment Distribution	3
1.2.4	Analysis of Word Frequencies by Sentiment	4
1.2.5	Text Length Analysis	4
1.3	Data Vectorization	5
1.3.1	Word-Embeddings	5
1.3.2	GloVE-Embeddings	5
1.3.3	Embeddings Exploration	5
2	Model Training and Evaluation	6
2.1	Model Foundations	6
2.1.1	Dummy Model	6
2.1.2	Base Model	6
2.1.3	Activation Functions	7
2.1.4	Deep Network and Data Regularization	7
2.1.5	Embeddings Dimension	8
2.1.6	Batch Normalization & Learning Rate Schedule	9
2.2	Hyperparameter Tuning	9
2.2.1	Optimizers	9
2.2.2	Loss Functions	10
2.2.3	Optuna Framework	10
2.3	Appendices	10
2.3.1	Other Embeddings	10
2.3.2	Preprocessing Modification	11
3	Conclusion	11
3.1	Experimental Summary	11
3.2	Comparison with the First Project	11
4	Bibliography	12

ABSTRACT

In this article, we humbly delve into the field of **Natural Language Processing (NLP)** by attempting to develop a machine learning model aimed at classifying **user comments** as either **positive** or **negative**. Our modest objective is to construct a basic yet functional **sentiment classifier**, utilizing a dataset composed of user-generated **Twitter** comments. For the purpose of data representation, we employ **word-embeddings**, and the model itself is implemented using the relatively straightforward architecture of **feed-forward neural networks**.

KEYWORDS. Machine Learning, Sentiment Classifier, Word Embedding, Feed Forward Neural Network

1 Data Processing and Analysis

The dataset is first **processed** and **cleaned**. In data analysis and machine learning, data preprocessing is considered the most critical initial phase of the workflow. Ensuring that the data is in a **suitable format** is essential for effective algorithmic applications.

1.1 Data Pre-processing

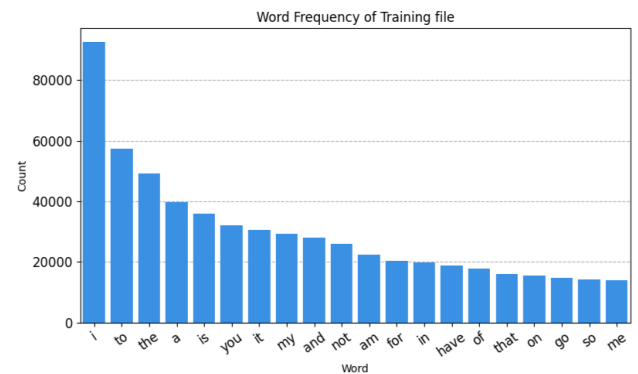
1.1.1. General Cleaning. The dataset underwent several preprocessing and cleaning techniques, including: 1) **Removal of URLs, user mentions, hashtags, special characters, and numbers**, 2) **Lowercasing all characters**, 3) **Elimination of extra whitespaces**, 4) **Stemming and lemmatization** of words, and 5) **Removal of punctuation**.

Applying these methods to the dataset yielded the results shown in Table 1, which displays a representative subset of transformed tweets. The table demonstrates how the preprocessing steps modified the text: user mentions were removed, characters were converted to lowercase, and other changes were applied.

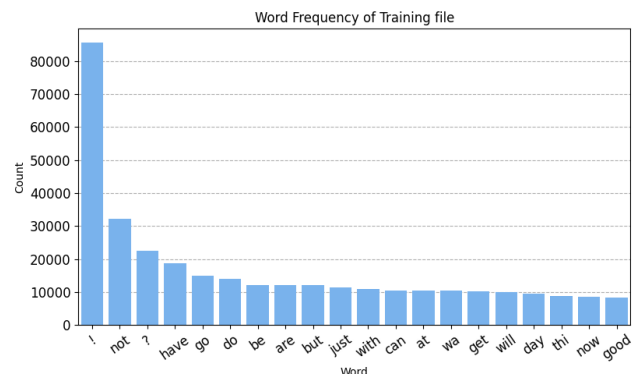
Notably, certain punctuation marks—**exclamation points (!)**, **question marks (?)**, and **ellipses (...)**—were intentionally retained due to their potential sentiment value. For instance, an **exclamation point (!)** may indicate enthusiasm, while **ellipses (...)** might suggest hesitation or irony.

1.1.2. Stop-words Removal. The subsequent preprocessing phase focuses on eliminating words with minimal sentiment value, known as **stop-words**. These terms provide negligible benefit for model training due to their limited semantic content. As such words frequently dominate text corpora without meaningful interpretation, their removal was prioritized.

A custom stop-word removal approach was implemented, differing from standard library-based methods. The processed dataset results are shown in Table 1, with the complete preprocessing workflow visualized in Figure 1 for the **training file**.



(a) General Cleaning word frequency



(b) Stop-words removal word frequency

Figure 1: Most frequent words across preprocessing steps

ID	Original Text	General Cleaning	Stop-words Removal
39015	how much i do love him	how much i do love him	how much do love him
15358	@Simpsdj Cool stuffs, sounds interesting	cool stuff sound interest	cool stuff sound interest
7666	@zackalltimelow dude what is it like	dude what is it like	dude what like
103061	on my way to METROBAR... teleserv anniv	on my way to metrobar... teleserv anniv	way metrobar... teleserv anniv

Table 1: Text Transformation after preprocessing. Second column shows the original text, the third column represents the text after general cleaning and fourth column shows the text after removing stop-words.

1.2 Exploratory Data Analysis

We will now proceed to some deeper analysis of the dataset.

1.2.1. Sentiment Correlation. The Exploratory Data Analysis (EDA) commenced with **sentiment correlation analysis** to identify words most frequently associated with each sentiment. The dataset was divided into positive and negative **subsets**, followed by extraction of frequent terms. The results are presented in Figure 2.

1.2.2. Dataset Splitting. In machine learning applications, proper dataset partitioning is essential for model development. The standard approach divides data into three distinct subsets: **70% for training**, **20% for validation**, and **10% for testing**. This structured division facilitates effective model training, hyperparameter tuning, and final evaluation. The current dataset's file size distribution across these splits is presented in Table 2, with additional visualization provided in Figure 3.

Dataset File	Records Count	Size Rate
Training	593.552	$\approx 70.00\%$
Validating	169.584	$\approx 20.00\%$
Testing	63.597	$\approx 10.00\%$

Table 2: Dataset training, validating and testing files size

1.2.3. Sentiment Distribution. A **Sentiment Distribution Analysis** was conducted for both training and validation datasets. This analysis quantified the proportion of positive versus negative comments in each dataset. The results, presented in Table 3 and Figure 3, reveal near-equal representation of both sentiment classes. While **positive comments** slightly **outnumber** negative ones in both datasets, the marginal difference confirms their **balanced** distribution. This equilibrium suggests neither sentiment class dominates the training or validation data.

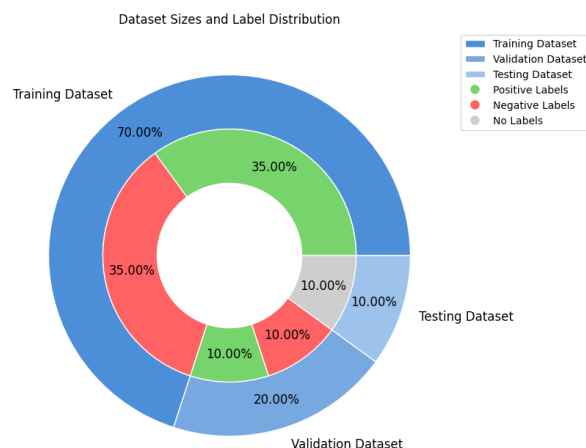


Figure 3: Training, Validating and Testing Datasets sizing and sentiment distribution analysis

Data Balance. The balanced dataset ensures a **fair learning process**, where the model is not biased toward any particular class due to unequal representation. This equilibrium promotes better generalization across all classes and yields more reliable validation performance. Consequently, evaluation metrics including **accuracy**, **precision**, **recall**, and **F1-score** provide meaningful assessments of the model's predictive capabilities.

Dataset File	Positive Tweets	Negative Tweets
Training	74.196	74.192
Validating	21.199	21.197
Testing	-	-

Table 3: Sentiment Distribution across the files. How many positive and negative tweets are there.

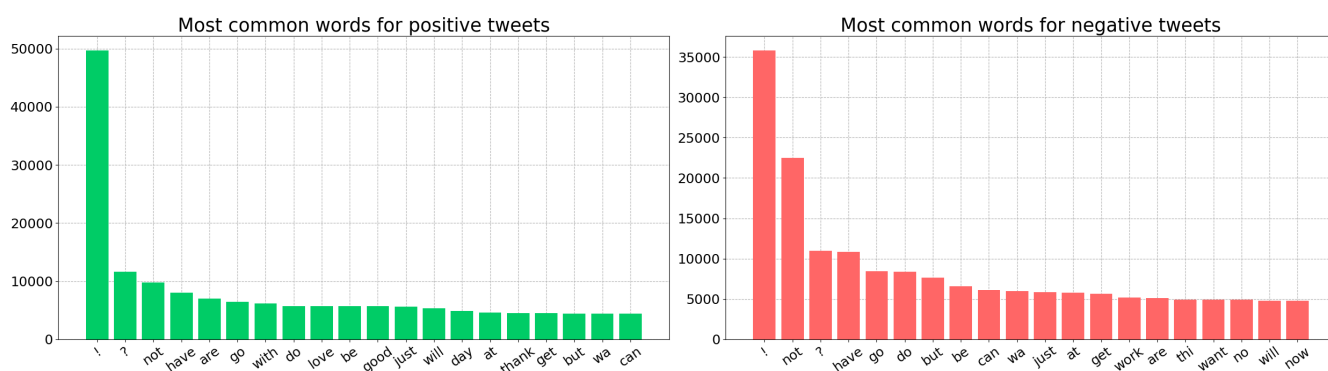


Figure 2: Sentiment Correlation: Most frequent words for positive and negative tweets file

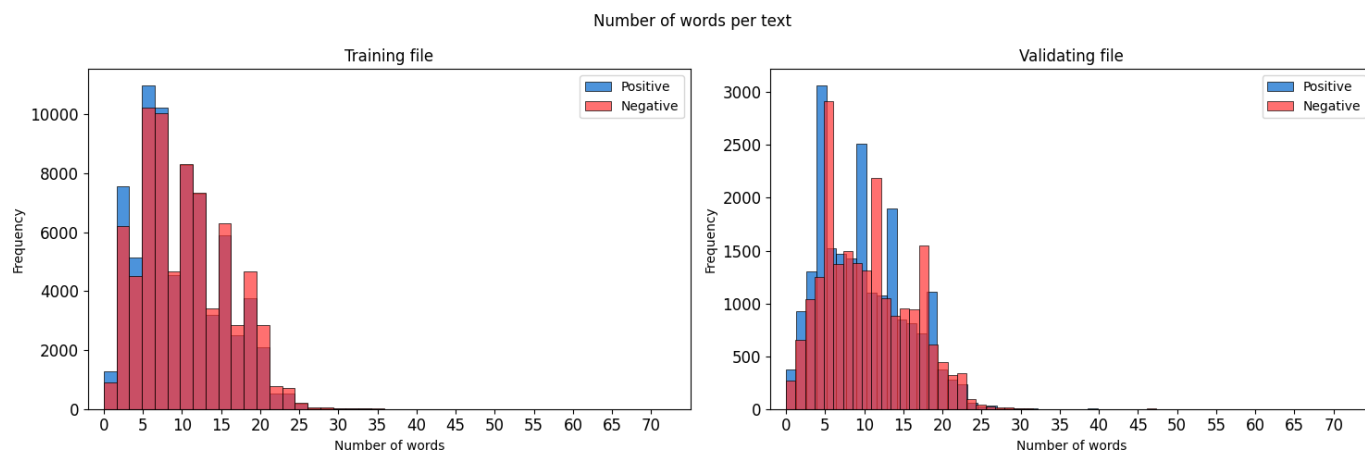


Figure 4: Sentiment Text Length Analysis for both training and validation files. X-axis represents the number of words and y-axis is the frequency of these numbers inside the files.

1.2.4. Analysis of Word Frequencies by Sentiment. Let's take a closer look at which words appear most often in our dataset and how they differ between positive and negative tweets. Figure 1 gives us an overview of the most common words overall, while Figure 5 specifically compares how these top 20 words appear in positive versus negative tweets.

Some interesting patterns stand out when we examine these results. Positive tweets tend to use certain features much more frequently - for example, **exclamation marks (!)** appear more often in positive tweets compared to negative ones. On the other hand, words that express negation like **'not'** show up significantly more in negative tweets. Other words like "good" and "great" naturally appear more in positive tweets, while terms like "bad" and "worst" dominate in negative ones. These patterns make perfect sense when we think about how people express positive versus negative feelings in their writing. The clear differences in word usage help our model learn to distinguish between these two sentiment categories.

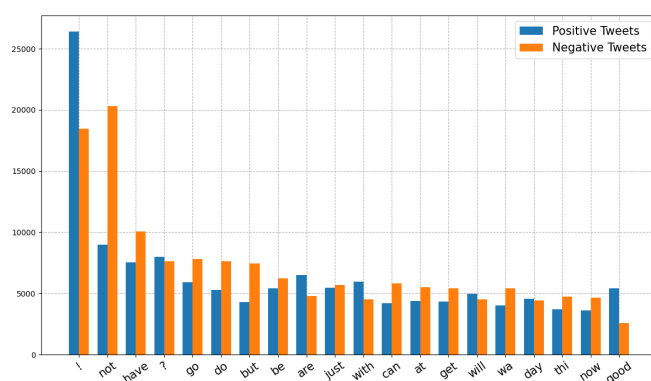


Figure 5: Comparative Analysis of Word Frequencies by Sentiment. How often does a word appear in positive and negative tweets.

1.2.5. Text Length Analysis. Now let's examine how long the tweets are in our dataset. We analyzed the number of words in both our **training set** (which contains 70% of our data) and our **validation set** (containing 20%). Looking at Figure 6, we can see that most tweets are quite short, with the majority falling between **0 to 30 words**.

In the training data, the most common tweet lengths are **5-6 words**, while in the validation set it's slightly shorter at **4-5 words**. The validation set shows smaller numbers overall, but this is exactly what we'd expect since it's a smaller sample of data. When we separate tweets by sentiment (shown in Figure 4), we notice that positive and negative tweets have almost identical length distributions. Whether someone is expressing something positive or negative doesn't seem to affect how long their tweet is - both types average around the same word count. This tells us that text length probably won't be a useful feature for our sentiment classifier, since it doesn't correlate with sentiment.

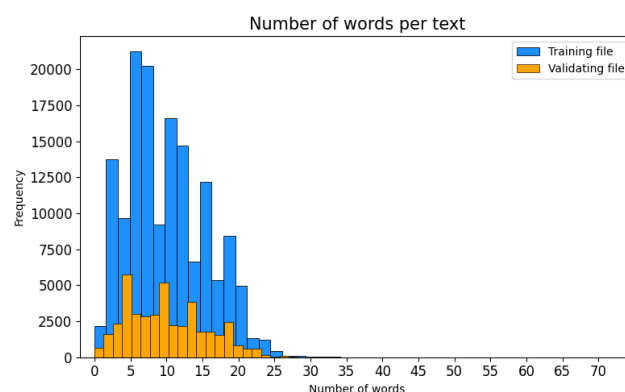


Figure 6: Text Length Analysis for training and validation dataset files. X-axis represents the number of words and y-axis is the frequency of these numbers inside the files.

1.3 Data Vectorization

In order to utilize our dataset within machine learning training procedures, it is imperative to first convert the textual data into numerical representations. Numerous vectorization techniques exist; however, in this work, we humbly concentrate on the use of **word-embeddings**, which offers a powerful yet accessible means of representing linguistic data.

1.3.1. Word-Embeddings. Word-embeddings constitute a widely adopted vectorization method in the domains of machine learning and, more specifically, natural language processing. The fundamental premise of this approach is that each word in a corpus can be mapped to a fixed-size numerical vector. These vectors are typically derived from extensive corpora and are thus often referred to as **pre-trained word-embeddings**. Fortunately, there exists a wealth of such embeddings publicly available online, many of which are extensively employed in modern deep learning applications.

1.3.2. GloVe-Embeddings. In our implementation, we modestly begin by employing the well-established GloVe-embeddings[1]. The GloVe dataset comprises hundreds of thousands of English words, each represented by a numerical vector (Figure 8). For our purposes, we utilize the **glove.6B.zip** dataset, which has graciously been made available for research and educational use.

This dataset is divided into four files, each offering a different vector dimensionality. Specifically:

1. glove.6B.50d.txt → 50-dimensional vectors
2. glove.6B.100d.txt → 100-dimensional vectors
3. glove.6B.200d.txt → 200-dimensional vectors
4. glove.6B.300d.txt → 300-dimensional vectors

While the choice among these options may seem trivial, we must admit that selecting an appropriate dimensionality is not always straightforward and may require empirical experimentation or, at the very least, cautious consideration. We start by using the 50-dimension representation of data.

1.3.3. Embeddings Exploration. GloVe embeddings are constructed in such a way that the resulting word vectors capture meaningful geometric relationships between words. To gain an intuitive understanding of this property, we examined the most similar words to a given term by calculating their proximity in vector space. Specifically, word similarity was computed using the **Euclidean distance**, which is a common approach when working with continuous vector representations.

Word	Similarity Score
technologies	0.8928267955780029
computer	0.8525559902191162
systems	0.8289326429367065
software	0.8089752197265625

Table 4: Words Similarity: Most similar words to *technology*

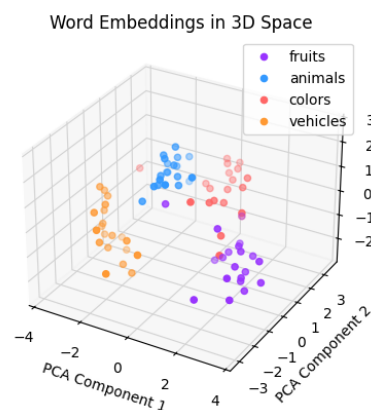


Figure 7: GloVe Embeddings words visualization in 3D space, using 50-dimension vectors (4 categories: fruits, animals, colors, vehicles)

As an illustrative example, Table 4 presents the words most similar to the term *technology*, according to the pre-trained embeddings. We acknowledge that this is a relatively simple exploration, but it offers a glimpse into the latent structure captured by the embeddings and reassures us that the vectors are not random, but rather encode semantic information.

$$\text{the} \rightarrow \begin{bmatrix} 0.418 \\ 0.249 \\ \vdots \end{bmatrix}, \text{ and } \rightarrow \begin{bmatrix} 0.268 \\ 0.143 \\ \vdots \end{bmatrix}, \dots \in \mathbb{R}^{50}$$

Figure 8: Word-Embeddings example: Representation of words in vector form

Another intriguing property of word embeddings is their ability to support arithmetic operations on word vectors. In other words, we can perform simple vector calculations that often yield semantically meaningful results. Although we acknowledge that such operations are not always perfectly interpretable, they can still provide valuable insight into the structure of the embedding space.

As a basic exploratory exercise, we computed the operations *apple* + *technology* and *apple* − *technology*. The resulting vectors were then mapped back to their closest words in the embedding space, as shown in Table 5. While the outcomes may not always align perfectly with intuitive expectations, they nevertheless suggest that the embeddings capture a degree of semantic logic that is both fascinating and useful.

$$\begin{aligned} \text{apple} + \text{technology} &= \text{software} \\ \text{apple} - \text{technology} &= \text{nectarine} \end{aligned}$$

Table 5: Word-Embeddings Operations: Arithmetic computations involving *apple* and *technology*

In order to gain a deeper understanding of the structure and geometric properties of the word embeddings, we attempted to visualize them in a three-dimensional space. As embedding vectors are originally situated in high-dimensional spaces, we

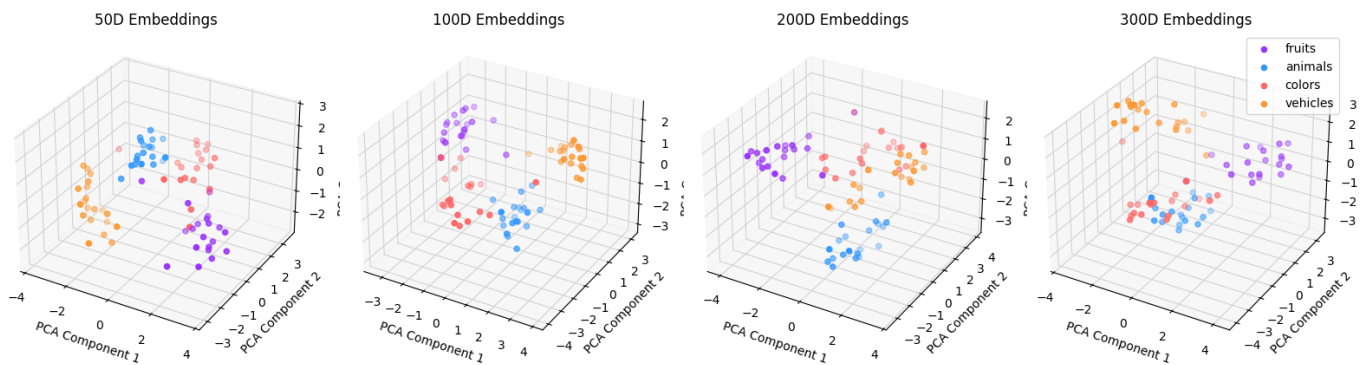


Figure 9: GloVe Embeddings words visualization in 3D space, using all vectors dimensions (4 categories: fruits, animals, colors, vehicles)

needed to apply dimensionality reduction techniques to make visualization feasible. A widely adopted method for such purposes is **Principal Component Analysis (PCA)**, which we employed to project the high-dimensional vectors into a more interpretable 3D space.

By applying PCA to our embeddings dataset, we successfully reduced the dimensionality of the vectors while striving to preserve as much of the original structure as possible. Figure 7 illustrates a selection of words from four semantic categories—fruits, animals, colors, and vehicles—based on the 50-dimensional embedding vectors. Additionally, Figure 9 presents a broader visualization, incorporating embeddings of all available dimensions (50, 100, 200, and 300). We acknowledge that such visualizations offer only an approximation of the underlying relationships, but they nonetheless provide an intuitive glimpse into the embedding space.

2 Model Training and Evaluation

With our data successfully transformed into numerical vector representations, we were finally able to proceed with the model training phase.

2.1 Model Foundations

2.1.1. Dummy Model. As a preliminary step—following standard practice and echoing our previous work—we began by evaluating a baseline model. This initial model, which we refer to as the *dummy model*, does not undergo any training and simply produces random predictions. Unsurprisingly, the accuracy achieved by this model hovers around 0.5, which is consistent with expectations given that our dataset is evenly balanced between two classes: *positive* and *negative*. While this model offers no predictive power, it provides a useful reference point against which we can measure the performance of our trained models. Its simplicity may seem trivial, but it plays a valuable role in contextualizing the results of more sophisticated approaches.

2.1.2. Base Model. We now proceed to implement a more structured model architecture, which we hoped would yield

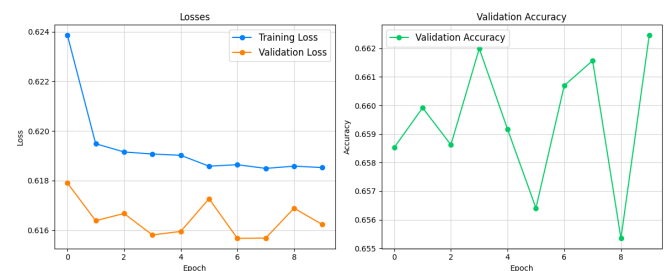


Figure 10: Base model training results. Training and validation loss values for 10 epochs of training.

improved performance over the baseline. At this stage, we introduce what we refer to as the *base model*, a straightforward neural network architecture depicted in Figure 11. This model consists of an **input layer** with 50 neurons—corresponding to the dimensionality of the word embeddings used—a single **hidden layer** with 128 neurons, and an **output layer** with 2 neurons representing the binary classification labels: *positive* and *negative*.

We trained this model using the following configuration: **10 epochs**, a **learning rate of 0.001**, the **Cross Entropy** loss function, and the **Adam** optimizer. The training process and evaluation results are summarized in Figure 10, where the left graph illustrates the evolution of both **training** and **validation loss**, and the right graph displays the trend of validation accuracy across epochs.

The final performance metrics obtained from evaluating the validation dataset are as follows:

- accuracy ≈ 0.6625
- precision ≈ 0.6634
- recall ≈ 0.6625
- f1-score ≈ 0.6620

The corresponding confusion matrix is presented in Table 6. Upon examining Figure 10, it becomes evident that the validation accuracy fluctuates considerably across epochs, indicating a lack of stability in the model's generalization capabilities. We suspect this may be attributed to the overly simplistic architecture and, notably, the absence of any activation functions

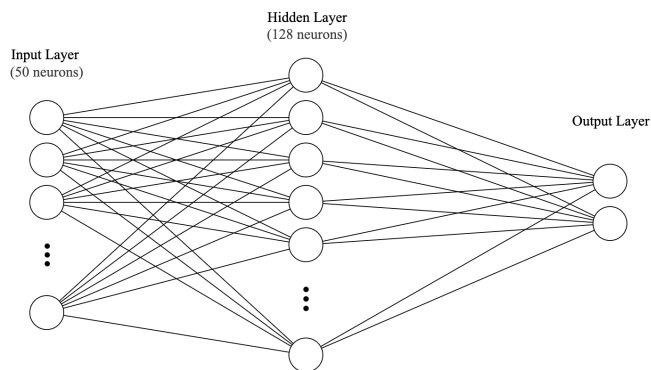


Figure 11: Base model neural feed forward network architecture (1 hidden layer - 128 neurons)

within the network. While we recognize that this model represents a step forward from the dummy baseline, it is clear that further refinement is necessary to achieve more reliable and consistent results.

True/Predicted Label	Negative (0)	Positive (1)
Negative (0)	14.840	6.357
Positive (1)	7.953	13.246

Table 6: Confusion Matrix of the Base Model

2.1.3. Activation Functions. In this section, we enhance our model architecture by incorporating an activation function. Activation functions play a crucial role in neural networks, as they enable the model to capture non-linear relationships and learn more complex patterns from the training data—ultimately improving its generalization capability.

PyTorch provides a variety of activation functions, among which the **ReLU** (Rectified Linear Unit) is one of the most widely used due to its simplicity and effectiveness. In our case, since the model currently contains only one hidden layer, we applied the ReLU activation function to that layer.

The results of this updated training process are presented in Figure 12. From these graphs, we observe a noticeable improvement in the model's behavior. Specifically, the training and validation loss curves begin to converge after a certain number of epochs, and the validation accuracy exhibits a more stable pattern—unlike the fluctuations seen in the earlier base model.

Despite these improvements in training dynamics, the final evaluation metrics were as follows:

- accuracy ≈ 0.6877
- precision ≈ 0.6878
- recall ≈ 0.6877
- f1-score ≈ 0.6877

Although these scores are slightly higher than those of the previous model, the overall training behavior suggests that

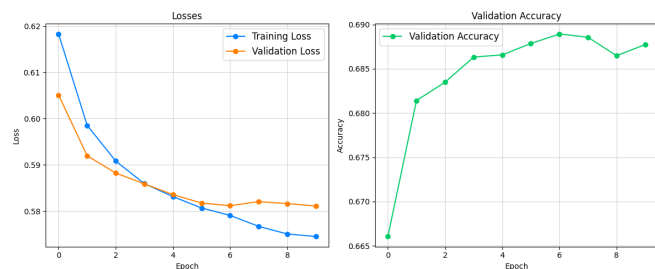


Figure 12: Training results after the addition of ReLU activation function to the hidden layer of the network (1 hidden layer - 128 neurons)

the model is beginning to generalize better. We take this as a small but important step in the right direction, and acknowledge that further tuning and architectural improvements may be necessary to realize the full potential of the network.

2.1.4. Deep Network and Data Regularization. The next step in our project involved constructing a deeper neural network to examine how increased model complexity influences training dynamics and evaluation performance. Specifically, we implemented the architecture illustrated in Figure 14, which consists of **four hidden layers** containing **256, 128, 64, and 32 neurons**, respectively.

This model was trained using the same hyperparameters as in previous experiments. The corresponding training results are presented in Figure 13. As evident from the plots, the model initially retains the generalization behavior observed earlier. However, signs of **overfitting** begin to emerge after the third epoch, as indicated by the growing gap between training and validation loss curves. This behavior suggested that the increased capacity of the network may be fitting the training data too closely, leading to diminished performance on unseen data.

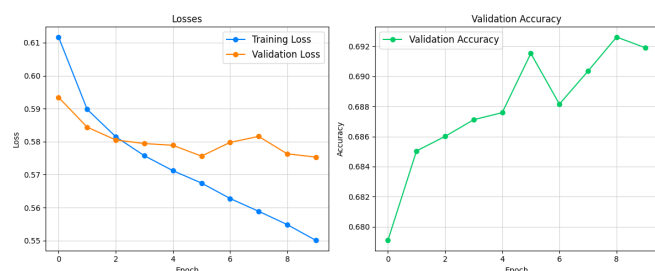


Figure 13: Training results of deep neural network architecture model (4 hidden layers - 256, 128, 64, 32 neurons respectively)

To address this issue, we applied **data regularization techniques**. First, we introduced **dropout** with a rate of **50%** in each hidden layer to mitigate co-adaptation of neurons. Additionally, we incorporated a small amount of **weight decay** (approximately 0.00001) into the optimizer to penalize overly complex weights. Finally, we increased the number of training epochs from 10 to 20 to allow the model more time to converge under regularized conditions.

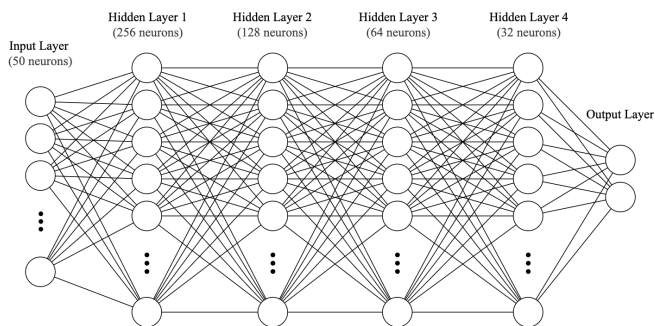


Figure 14: Deep model neural feed forward network architecture (4 hidden layers - 256, 128, 64, 32 neurons respectively)

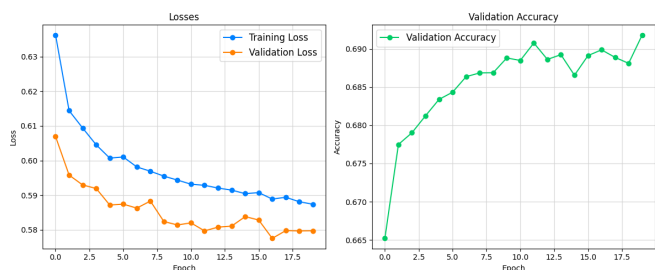


Figure 15: Training results of deep neural network model (4 hidden layers - 256, 128, 64, 32 neurons respectively), Data Regularization (50% dropout, 0.00001 of optimizer weight decay)

These modifications helped to reduce the overfitting effect. As shown in Figure 15, the validation loss remained more closely aligned with the training loss, and the two began to converge again—indicating improved generalization. While we understand that these adjustments do not eliminate overfitting entirely, they mark a significant improvement in model robustness and training stability.

2.1.5. Embeddings Dimension. Up to this point, we have used 50-dimensional GloVe vectors to represent words. However, as previously discussed, GloVe provides several options for embedding dimensionality: 50, 100, 200, and 300 dimensions. Intuitively, one might expect that increasing the dimensionality of word embeddings should enhance their descriptive power—allowing the model to capture more nuanced semantic relationships. Yet, this is not always the case, and increasing dimensionality can also introduce drawbacks such as overfitting or unnecessary model complexity.

In this section, we explore how embedding dimensionality affects model performance. We trained four separate versions of our model using each of the available GloVe vector dimensions. The resulting training and validation losses across epochs are shown in Figure 16.

Interestingly, although the 200-dimensional embeddings yielded better convergence behavior than the 300-dimensional ones (especially in terms of training dynamics) we ultimately favored the 300-dimensional representation. As demonstrated in Figure 17, this higher-dimensional embedding consistently

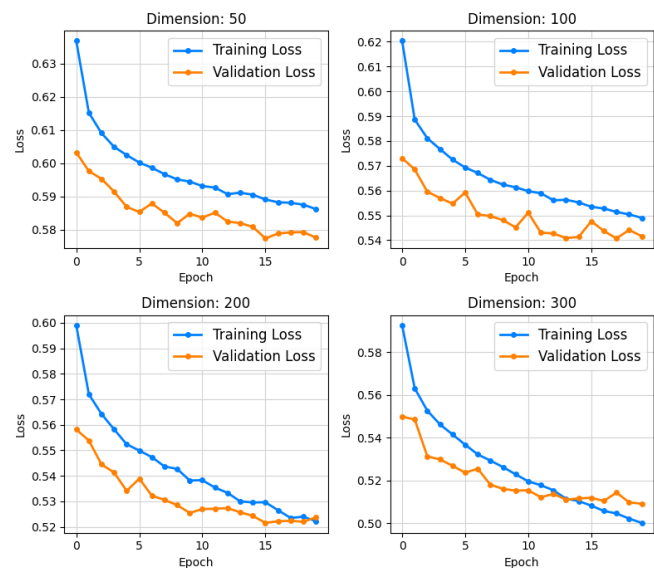


Figure 16: Training results of the deep neural network model, with Data Regularization for every GloVe Embedding vector dimension

resulted in improved validation accuracy across the board. This finding confirms our earlier hypothesis: in this specific case, richer embeddings lead to better model performance.

However, we did observe a degree of **minor overfitting** with the 300-dimensional vectors, evidenced by a growing gap between the training and validation loss curves in later epochs. While this divergence is more pronounced compared to the 200-dimensional model, it remains relatively small and does not pose a significant concern for model generalization.

Through these experiments, we also identified that the model performs best when trained for **20 epochs**, as this is the point at which the overfitting tendency starts to emerge. Thus, we consider this configuration to represent the most balanced and effective version of our model thus far.

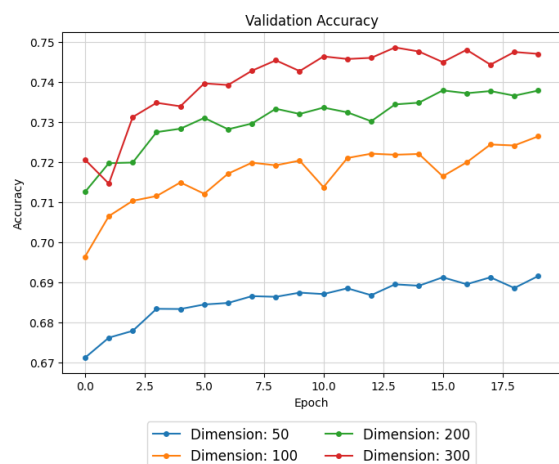


Figure 17: Evaluation results (validation accuracy values) of the deep neural network, with Data Regularization for every GloVe Embedding vector dimension

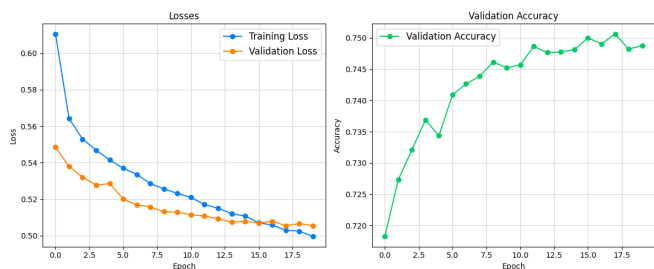


Figure 18: Training results of deep neural network with 4 hidden layers (256, 128, 64, 32 neurons), data regularization (50% dropout, optimizer weight decay = 0.00001), batch normalization (one-dimensional across 5 layers), and learning rate schedule (step_size = 2, gamma = 0.5).

2.1.6. Batch Normalization & Learning Rate Schedule. As the final refinement to our model architecture, we incorporated two widely used techniques: **batch normalization** and a **learning rate scheduler**. These additions were aimed at improving the training dynamics and potentially enhancing the overall performance of the model.

Batch normalization was applied between the layers of the network to stabilize and accelerate training by normalizing the input of each layer. Specifically, we inserted one-dimensional batch normalization layers after each linear transformation. This helped reduce internal covariate shift and contributed to smoother convergence.

In parallel, we introduced a learning rate schedule using PyTorch’s StepLR scheduler. The learning rate was configured to decay by a factor of 0.5 every 2 epochs. In other words step_size=2 and gamma=0.5 were used in this step. This dynamic adjustment allows the model to take larger steps in the early phases of training and gradually fine-tune its parameters as convergence is approached.

The training results obtained from this final configuration are presented in Figure 18. As shown, the combination of regularization (dropout and weight decay), batch normalization, and learning rate scheduling resulted in more stable training behavior and better convergence, particularly in the later stages of training.

True/Predicted Label	Negative (0)	Positive (1)
Negative (0)	16.215	4.982
Positive (1)	5.668	15.531

Table 7: Confusion Matrix of the deep neural network with 4 hidden layers (256, 128, 64, 32 neurons), data regularization (50% dropout, optimizer weight decay = 0.00001), batch normalization (one-dimensional across 5 layers), and learning rate scheduling (step_size = 2, gamma = 0.5).

The evaluation metrics achieved by this final model configuration are as follows: **accuracy** = 0.7488, **precision** = 0.7491, **recall** = 0.7488, and **f1-score** = 0.7487. These results indicate that the model maintained a solid and balanced performance across key evaluation criteria. The corresponding confusion matrix, shown in Table 7, provides further insight into the distribution of predicted versus actual labels, demonstrating a rel-

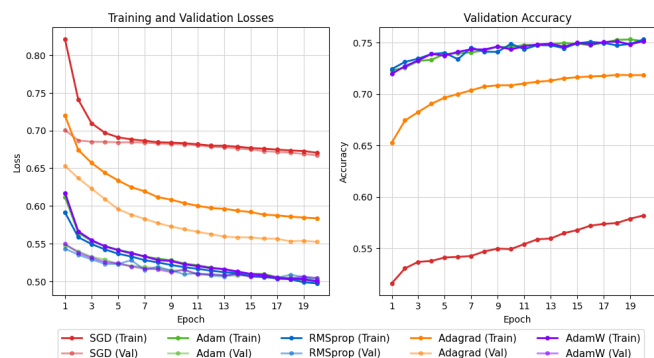


Figure 19: Training and Evaluation results of deep neural network for different optimizers (SGD, Adagrad, Adam, AdamW, RMSprop)

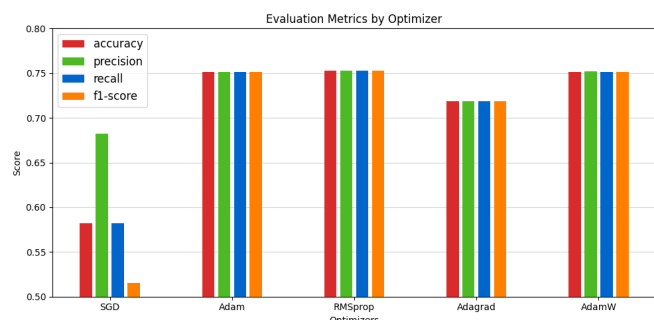


Figure 20: Evaluation metric results (Accuracy, Precision, Recall, F1-score) of deep neural network for different optimizers (SGD, Adagrad, Adam, AdamW, RMSprop)

atively even classification pattern across both sentiment categories.

2.2 Hyperparameter Tuning

Having now explored and implemented the fundamental components of a feed-forward neural network, the time has come to address one of the most delicate and, at times, frustrating aspects of model development—hyperparameter tuning. In this section, we attempt to refine our model’s performance by experimenting with various hyperparameters, hoping to coax out even marginal gains.

2.2.1. Optimizers. Our journey begins with the exploration of different optimization algorithms. We trained separate versions of our model using a range of optimizers, including **SGD**, **Adam**, **RMSprop**, **Adagrad**, and **AdamW**. The results of these experiments are presented in Figure 19. This figure displays the training and validation loss curves on the left—where brighter lines denote training loss and lighter ones represent validation loss—while the right-hand side illustrates the validation accuracy across epochs for each optimizer.

From these plots, a somewhat disheartening but unsurprising observation emerges: **SGD** and **Adagrad** significantly underperform, showing higher losses and notably lower validation accuracies. Meanwhile, **Adam**, **AdamW**, and **RMSprop**

Opt. / Metric	Accuracy	Precision	Recall	F1-Score
SGD	0.5819	0.6820	0.5819	0.5152
Adagrad	0.7183	0.7184	0.7183	0.7183
Adam	0.7513	0.7516	0.7513	0.7512
AdamW	0.7512	0.7517	0.7512	0.7511
RMSprop	0.7530	0.7531	0.7530	0.7529

Table 8: Analytic evaluation metric results (Accuracy, Precision, Recall, F1-score) of deep neural network for different optimizers (SGD, Adagrad, Adam, AdamW, RMSprop)

show much more promising results, with better convergence behavior and more stable accuracy trajectories. However, determining a clear winner among these three remains elusive when relying solely on visual inspection.

Thankfully, Figure 20 and Table 8 offer a more detailed breakdown of the evaluation metrics. These results reinforce our earlier impression: **SGD** and **Adagrad** yield the lowest performance, while the other three optimizers maintain similar, and generally superior, metric values. Notably, **RMSprop** slightly edges out its competitors with a validation accuracy of **0.7530**, thereby earning our preference for use in subsequent experiments. Though the gains are modest, they offer a much-needed sense of progress in an otherwise incremental tuning process.

2.2.2. Loss Functions. Another hyperparameter that warrants investigation is the model’s **loss function**. The choice of loss function can significantly influence training dynamics, depending on the nature of the classification task. While **Cross Entropy Loss** is a standard for multiclass classification problems, binary classification tasks such as ours sometimes benefit from alternative options like **BCELoss**. To this end, we evaluated three loss functions (**Cross Entropy**, **MSELoss**, and **BCELoss**) to assess their comparative impact on performance.

It is worth noting that to properly utilize **MSELoss** and **BCELoss**, we had to slightly adjust our model architecture from that shown in Figure 14. Specifically, the final output layer was modified to consist of a single neuron, suitable for binary output, with a **sigmoid** activation function applied to produce probabilities. Despite this change, the evaluation approach remained consistent with previous experiments.

Figure 21 illustrates the loss and validation accuracy for each loss function. While the left-hand graph appears to suggest that **MSELoss** results in significantly lower loss values, this is more a reflection of how the loss is computed than an actual improvement in model performance. In reality, the convergence behavior of both training and validation loss is nearly identical across all three loss functions. Likewise, the validation accuracy shows similar trends in all cases, fluctuating modestly and without a clear advantage for any specific loss function.

In conclusion—somewhat anticlimactically—there is no significant performance difference tied to the choice among these

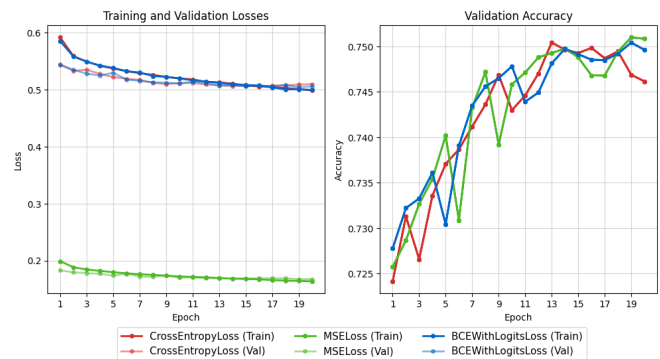


Figure 21: Training and Evaluation results of deep neural network for different loss functions (Cross-Entropy, BCELoss, MSELoss)

three loss functions in our setup. As such, we opt to continue using **Cross Entropy Loss**, if only for consistency and conventionality.

2.2.3. Optuna Framework. As a final experiment, we turned our attention to the remaining hyperparameters of the model namely, the number of layers, neurons per layer, dropout rate, learning rate, batch size, and weight decay. Given the sheer number of combinations and the complexity involved in tuning them manually, it quickly became clear that a more systematic approach was necessary. To this end, we employed the **Optuna** framework, a modern and efficient solution for hyperparameter optimization. This framework allowed us to conduct a focused search through the vast hyperparameter space with minimal manual intervention.

By setting up a custom Optuna study, we ran five trials, each testing different combinations of the aforementioned parameters. Although five trials is admittedly a small sample (reflecting, perhaps, both resource constraints and a faint sense of resignation) the framework nonetheless identified a promising configuration, detailed below:

- number of layers = 4
- neurons per layer = 288
- dropout rate = 35%
- learning rate = 0.0003681
- batch size = 32
- weight decay = 0.00007871

This specific set of hyperparameters yielded a validation accuracy of **0.7580**, which, while not groundbreaking, represents the highest result achieved in our experiments. At the very least, it offers a glimmer of hope that automated tuning can push performance beyond the limitations of trial-and-error guesswork.

2.3 Appendices

2.3.1. Other Embeddings. Until now, our vectorization efforts have been solely focused on GloVe embeddings. While GloVe has served us well, it would be remiss not to test alternative

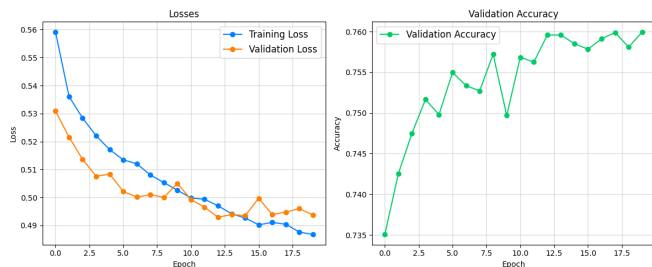


Figure 22: Training and Evaluation results of deep neural network using the GoogleNews-negative-300 embeddings

embedding schemes—if only to say we did. Since the core methodology of this project revolves around word embeddings, we decided to explore another popular source: the **GoogleNews-negative-300** embeddings. This library offers word vectors much like GloVe, but exclusively in a 300-dimensional format. In other words, fewer choices, less decision fatigue.

We ran the same model architecture using the hyperparameters previously discovered through the Optuna framework (in our glorious five-trial experiment). The results of this experiment are shown in Figure 22. On casual inspection, the performance appears marginally better—validation loss trends slightly downward, and accuracy flirts with improvement. Specifically, the best accuracy we observed reached **0.76**, which (while not revolutionary) does technically represent an improvement.

For the sake of completeness (and closure), the full set of evaluation metrics are shown below:

- Accuracy = 0.76
- Precision = 0.7602
- Recall = 0.76
- F1-score = 0.7599

2.3.2. Preprocessing Modification. Finally, echoing the findings of our previous project, we revisited the impact of classic preprocessing techniques—namely, stop-word removal, stemming, and lemmatization. While conventional wisdom would suggest these steps help refine input data, our past results hinted otherwise. In the spirit of scientific repetition (and mild disbelief), we once again tested the scenario in which we **skip** all three.

Surprisingly—but perhaps no longer shockingly—removing these preprocessing steps led to better performance. The training and validation losses, along with the accuracy improvements, are illustrated in Figure 23. The model achieved its best accuracy yet: **0.7730**.

The final evaluation metrics are summarized below:

- Accuracy = 0.7730
- Precision = 0.7744
- Recall = 0.7730
- F1-score = 0.7727

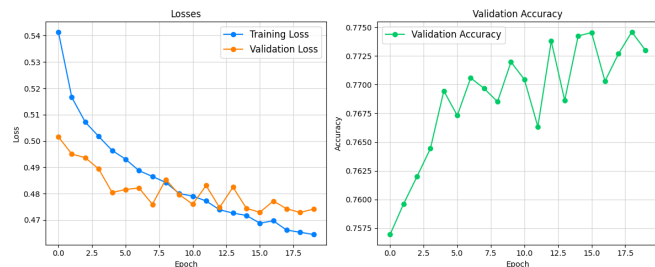


Figure 23: Training and evaluation results of the deep neural network after modifying the preprocessing pipeline (no stop-word removal, no stemming, no lemmatization).

3 Conclusion

3.1 Experimental Summary

Throughout this project, we systematically developed and refined a binary classification model using word embeddings. Starting from a simple untrained dummy model, we incrementally improved the architecture and training procedure through a series of thoughtful modifications and experiments.

We began with the use of pre-trained GloVe embeddings, testing basic architectures with and without activation functions. We then explored deeper neural networks and employed regularization techniques such as dropout and weight decay to mitigate overfitting. Further enhancements included batch normalization, learning rate scheduling, and extensive hyperparameter tuning using the Optuna framework.

We also evaluated the impact of word embedding dimensionality, compared different optimizers and loss functions, and validated our assumptions with alternative embedding sources (e.g., GoogleNews-negative-300). Finally, we revisited the preprocessing steps and found that removing stop-word deletion, stemming, and lemmatization significantly improved model performance.

The progression of our model’s performance is summarized in Table 9. As the table demonstrates, each experiment contributed toward improving the overall performance of our model. In particular, the surprising effectiveness of skipping certain preprocessing steps led to our best results. This highlights the importance of empirical validation in model development, even when intuition or standard practices suggest otherwise.

Our final model—using GoogleNews embeddings, Optuna (optimized hyperparameters, and minimal preprocessing) achieved an accuracy of **0.7730**, marking a substantial improvement over the baseline and offering a solid foundation for future enhancements.

3.2 Comparison with the First Project

When comparing the results of this project to our previous work using TF-IDF vectorization and Logistic Regression, we observe that the overall performance was slightly better in the earlier approach. Specifically, the best accuracy obtained in the

Model Version	Accuracy	Precision	Recall	F1-Score
Dummy Model	0.5000	0.5000	0.5000	0.5000
Base Model (No Activation)	0.7505	0.7508	0.7505	0.7505
Base Model + ReLU	0.6877	0.6878	0.6877	0.6877
Deep Network + Regularization	0.7488	0.7491	0.7488	0.7487
Deep Network + BatchNorm + LR Scheduler	0.7488	0.7491	0.7488	0.7487
Hyperparameter Tuning (Optuna)	0.7580	0.7587	0.7580	0.7578
GoogleNews Embeddings	0.7600	0.7607	0.7601	0.7594
Final Preprocessing Modification	0.7730	0.7744	0.7730	0.7727

Table 9: Summary of metric values across model development stages.

first project reached approximately **0.80**, whereas the highest accuracy achieved in this project was **0.7730**, despite extensive tuning and use of pre-trained embeddings.

This outcome highlights an important aspect of machine learning workflows: model architecture and training strategies alone may not always outperform simpler approaches if the data representation and preprocessing are not well aligned with the task. While deep learning offers powerful capabilities, its effectiveness is highly sensitive to data quality and representation.

It is also worth noting that in both projects, we observed a significant impact from preprocessing strategies. In fact, the final performance improvement in this project came from reversing standard preprocessing techniques (e.g., keeping stop words, avoiding stemming and lemmatization), which reinforces the idea that optimal preprocessing can be highly task-dependent.

Ultimately, while this project explored more complex models, the previous project’s simpler pipeline—when combined with effective preprocessing—proved to be more robust for the task at hand.

4 Bibliography

References

- [1] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. Glove: Global vectors for word representation. In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.